

PedEasy — Instruções de Deploy

Arquitetura: backend Node.js + SQLite por tenant + frontend React (Vite) + Docker

1. Pré-requisitos

- Git
- Docker + Docker Compose
- Um VPS ou servidor com Docker instalado (Linux recomendado)
- Um domínio (opcional, para produção)
- Conta no Vercel para o frontend (opcional)

2. Clonar o repositório

```
git clone https://github.com/gustavosadoque19-ctrl/PedEasy1.git
cd PedEasy1
```

3. Configurar variáveis de ambiente

Crie um arquivo .env na raiz do projeto:

```
# Obrigatórias
JWT_SECRET=coloque-uma-chave-secreta-forte-aqui

# Frontend (URL pública do backend para o Vercel)
# VITE_API_URL=https://api.seudominio.com/api

# Pagar.me (para processar pagamentos)
PAGARME_SECRET_KEY=sk...
PAGARME_PLAN_PRO=plan...
PAGARME_PLAN_ENTERPRISE=plan...

# Focus NFe (para emissão de notas fiscais)
FOCUS_TOKEN=...
FOCUS_CNPJ=...
FOCUS_CIDADE=Sao Paulo
FOCUS_UF=SP

# WhatsApp Bot
BOT_API_KEY=...
```

4. Subir com Docker Compose

O docker-compose.yml já está configurado para subir 3 serviços:

- frontend (nginx porta 80)
- backend (Express porta 3000)
- bot (WhatsApp porta 3001)

```
docker compose up -d --build
```

Verificar se os containers estão rodando:

```
docker compose ps
```

Ver logs do backend:

```
docker compose logs -f backend
```

5. Executar o Seed (primeira execução)

Após subir os containers, execute o seed para criar o primeiro tenant e dados iniciais:

```
docker compose exec backend npm run seed
```

Isso cria:

- Tenant "Estabelecimento Padrão" (admin@pedy.com / admin)
- Usuário administrador no banco global

- Banco SQLite do tenant com produtos, clientes e funcionários de exemplo

6. Acessar o sistema

- Frontend: `http://localhost` (porta 80)
- Backend API: `http://localhost:3000/api`
- Login SaaS: `admin@pedy.com` / `admin`

7. Deploy do Frontend no Vercel (opcional)

Se quiser usar o Vercel apenas para o frontend:

- Conecte o repositório no Vercel
- Em Settings !' Environment Variables, adicione:

`VITE_API_URL=https://seudominio.com/api`

- Faça o deploy (a main já está configurada com `vercel.json` para SPA)
- O backend precisa estar rodando em um servidor acessível publicamente

8. Novo tenant (cadastro)

Pelo formulário de signup do SaaS, um novo tenant é criado com:

- Registro no banco global (tenants + tenant_users)
- Criação automática de um novo arquivo SQLite (tenant-{id}.db)
- Todas as tabelas são criadas via `schema.sql`
- O tenant já fica pronto para uso imediato

9. Backup dos dados

Os bancos SQLite ficam em `backend/data/`. Para backup:

```
docker compose exec backend tar -czf /tmp/backup-data.tar.gz -C /app/data .
docker compose cp backend:/tmp/backup-data.tar.gz ./backup-$(date +%Y%m%d).tar.gz
```

10. Comandos úteis

Reconstruir e reiniciar:

```
docker compose down
docker compose up -d --build
```

Ver logs:

```
docker compose logs -f backend
```

Executar comandos no backend:

```
docker compose exec backend npm run seed
```

Acessar o banco SQLite diretamente:

```
docker compose exec backend sqlite3 /app/data/tenant-1.db
.tables
SELECT * FROM produtos;
```

% %

Documentação gerada em 17/06/2026, 22:55:03